



计算机应用
Journal of Computer Applications
ISSN 1001-9081, CN 51-1307/TP

《计算机应用》网络首发论文

题目：基于边权重的软件漏洞检测方法
作者：于巧，黄子睿，程圣懿，祝义，张淑涛
收稿日期：2025-03-06
网络首发日期：2025-05-20
引用格式：于巧，黄子睿，程圣懿，祝义，张淑涛. 基于边权重的软件漏洞检测方法[J/OL]. 计算机应用. <https://link.cnki.net/urlid/51.1307.TP.20250519.1734.006>



网络首发：在编辑部工作流程中，稿件从录用到出版要经历录用定稿、排版定稿、整期汇编定稿等阶段。录用定稿指内容已经确定，且通过同行评议、主编终审同意刊用的稿件。排版定稿指录用定稿按照期刊特定版式（包括网络呈现版式）排版后的稿件，可暂不确定出版年、卷、期和页码。整期汇编定稿指出版年、卷、期、页码均已确定的印刷或数字出版的整期汇编稿件。录用定稿网络首发稿件内容必须符合《出版管理条例》和《期刊出版管理规定》的有关规定；学术研究成果具有创新性、科学性和先进性，符合编辑部对刊文的录用要求，不存在学术不端行为及其他侵权行为；稿件内容应基本符合国家有关书刊编辑、出版的技术标准，正确使用和统一规范语言文字、符号、数字、外文字母、法定计量单位及地图标注等。为确保录用定稿网络首发的严肃性，录用定稿一经发布，不得修改论文题目、作者、机构名称和学术内容，只可基于编辑规范进行少量文字的修改。

出版确认：纸质期刊编辑部通过与《中国学术期刊（光盘版）》电子杂志社有限公司签约，在《中国学术期刊（网络版）》出版传播平台上创办与纸质期刊内容一致的网络版，以单篇或整期出版形式，在印刷出版之前刊发论文的录用定稿、排版定稿、整期汇编定稿。因为《中国学术期刊（网络版）》是国家新闻出版广电总局批准的网络连续型出版物（ISSN 2096-4188，CN 11-6037/Z），所以签约期刊的网络版上网络首发论文视为正式出版。

基于边权重的软件漏洞检测方法

于巧¹, 黄子睿^{1*}, 程圣懿¹, 祝义¹, 张淑涛²

(1. 江苏师范大学计算机科学与技术学院, 江苏 徐州 221116; 2. 江苏徐矿能源股份有限公司, 江苏 徐州 220009)

(* 通信作者电子邮箱 hanzir13@163.com)

摘要:随着软件在各个领域的广泛应用,软件漏洞呈现出不断增长趋势,基于深度学习的软件漏洞检测方法得到广泛应用。然而,现有的图表示学习方法往往忽略了图中边对软件漏洞检测的影响,对边权重的表示过于粗糙。针对该问题,提出了一种基于边权重的软件漏洞检测方法(Edge Weight for Vulnerability Detection, EWVD)。首先,对源代码中的注释、自定义变量名和函数名进行清理和抽象表示。其次,经过对比使用 Sent2vec 进行嵌入表示。再次,利用连接结构、邻居节点的重要性和 Jaccard 相似性三种度量方式,对边权重进行综合计算,从而识别节点间的信息传递能力。最后,模型利用边权重提升对漏洞语句潜在关系的感知能力,判断图中边的重要性。与七种漏洞检测基线方法中的最优基线 VulCNN 相比,本文提出的 EWVD 方法的 Accuracy 提高 1.06 个百分点, FPR 降低 1.11 个百分点。因此,EWVD 细化了边权重的表示,进行漏洞检测时综合性能得到提升。

关键词:漏洞检测;边权重;图表示学习

中图分类号:TP311 **文献标志码:**A

Software vulnerability detection method based on edge weight

YU Qiao¹, HUANG Zirui^{1*}, CHENG Shengyi¹, ZHU Yi¹, ZHANG Shutao²

(1. School of Computer Science and Technology, Jiangsu Normal University, Xuzhou Jiangsu 221116, China;

2. Jiangsu Xukuang Energy Co., Ltd., Xuzhou Jiangsu 220009, China)

Abstract: With the widespread application of software across various domains, software vulnerabilities have shown a continuous upward trend. Deep learning-based methods for vulnerability detection have gained wide adoption. However, existing graph representation learning methods were often found to neglect the influence of edges in the graph on vulnerability detection, and the representation of edge weights was overly coarse. To address this issue, a software vulnerability detection method based on edge weights (Edge Weight for Vulnerability Detection, EWVD) was proposed. Firstly, comments, user-defined variable names, and function names in the source code were cleaned and abstractly represented. Secondly, Sent2vec was used to generate embedding representations through comparative analysis. Thirdly, edge weights were comprehensively calculated using three metrics: connection structure, the importance of neighboring nodes, and Jaccard similarity, in order to identify the information transmission capability between nodes. Finally, the model was enhanced by leveraging edge weights to better perceive potential relationships between vulnerable statements and to determine the importance of edges in the graph. Compared with the best-performing baseline method VulCNN among seven vulnerability detection baselines, the proposed EWVD method achieved an increase of 1.06 percentage points in Accuracy and a decrease of 1.11 percentage points in FPR. Therefore, EWVD refined the representation of edge weights and improved the overall performance of vulnerability detection.

Key words: vulnerability detection; edge weight; graph representation learning

0 引言

随着信息技术的迅猛发展和软件应用的普及,软件漏洞的威胁日益严重。从近年的通用漏洞枚举^[1](Common Vulnerability Enumeration, CVE)来看,软件漏洞的数量正在迅速增加,已成为影响软件漏洞的一个重要因素。这些漏洞是软件开发过程中不合理的开发规范或疏忽的结果,一旦这些漏洞被恶意攻击者发现并利用,不仅

威胁着个人隐私的泄露,还可能对企业的财务和声誉造成严重损害,甚至影响国家安全。因此,软件漏洞的严重性和普遍性使得开发准确的漏洞检测技术变得至关重要。

传统的漏洞检测模型大致可以分为两大类:基于规则的漏洞检测模型和基于代码相似度的漏洞检测方法。传统的基于规则的漏洞检测方法^[2-4]通常依赖于专家手动定义的规则来检测漏洞。这些方法需要耗费大量的时间和资源,难以应对现代复杂软件系统中的大

收稿日期:2025-03-06;修回日期:2025-05-11;录用日期:2025-05-13。

基金项目:国家自然科学基金项目(61902161);江苏师范大学研究生科研与实践创新计划项目(2024XKT2616)。

作者简介:于巧(1989—),女,山东莱阳人,副教授,博士,CCF会员,主要研究方向:机器学习、软件缺陷预测、漏洞检测;黄子睿(1999—),男,江苏淮安人,硕士研究生,CCF学生会会员,主要研究方向:漏洞检测;程圣懿(1999—),男,江苏徐州人,硕士研究生,CCF学生会会员,主要研究方向:漏洞检测、软件缺陷预测;祝义(1976—),男,江西九江人,教授,博士,CCF高级会员,主要研究方向:软件工程、形式化方法、软件缺陷预测;张淑涛(1979—),男,江苏徐州人,高级工程师,硕士,主要研究方向:信息管理系统与运维。

量代码,并且很难同时实现低假阳性率(False Positive Rate, FPR)和低假阴性率(False Negative Rate, FNR)。此外,基于规则的检测模型依赖于已知的漏洞模式,随着漏洞的表现形式也越来越多样化和隐蔽化,这种方法难以识别新出现的、未知的漏洞。基于代码相似度的方法^[5-7]仅通过目标代码与脆弱代码之间的相似度比较来检测克隆漏洞,无法处理代码的语法和语义差异。这种方法误报率较高,尤其是当代码结构相似但语义不同的情况下。

近年来,深度学习技术在自然语言处理、图像处理、语音识别和智能驾驶等诸多领域取得了重大进展。同时,深度学习方法也被广泛用于使用检测源代码漏洞,除了提高漏洞检测的准确性和效率外,这些工具还可以通过学习大量的代码样本来自动识别潜在的漏洞,从而减少了特征提取的工作量,使专家从手动定义漏洞特征的繁琐任务中解脱出来。因此,对源代码进行自动检测和分析已成为软件工程领域的一个重要研究方向。

由于源代码和自然语言在某种程度上是相似的,一些研究者尝试用自然语言处理技术将源代码作为文本进行处理。但是这些基于文本的方法的检测性能并不理想,因为扁平的序列化特征不可避免地会丢失代码语义中的层次和结构信息,因此基于图的神经网络是一种更加理想的选择^[8]。基于图的模型使用不同类型的图来表示代码,例如抽象语法树(Abstract Syntax Tree, AST)、数据流图(Data Flow Graph, DFG)、控制流图(Control Flow Graph, CFG)等^[9]。这些基于图的方法在表示程序依赖性和语义方面非常重要。然而,它们通常包含许多与漏洞无关的程序实体及其依赖项,同时对图中与漏洞语句相关的无加权边的关注较低,并不能帮助模型更好地识别漏洞语句的潜在关系。

基于图的方法主要是根据源代码节点和边构成的网络,在本文中将重点探究网络中边的重要性。本文的关键思想受Nandi等人^[10]的启发,其提出的方法在复杂网络中使用边权重识别关键的病毒传播者具有显著作用。本文对该方法进行改进,利用边权重分析网络中具有影响力的传递信息节点,进而检测漏洞。

本文为了衡量节点信息的扩散潜力,探讨了节点之间的结构连通性。通过实验发现,每当相邻节点彼此连接时,信息的传递潜力就会增加,特别是当节点与其邻居形成三角形结构时,这种信息传递能力显著增强^[11]。因此,在衡量节点的信息扩散潜力时,引入了节点三角形结构的比例。

此外,节点的影响力在很大程度上依赖于其邻居的传递能力。每当一个节点连接到更重要的邻居时,它的影响力也会随之增强。为了有效地识别邻居节点的重要性,本文从局部和全局两个角度对邻居的拓扑信息进行评估。通过结合邻居节点的度中心性和接近中心性,可以更准确地识别邻居节点的重要性。

Jaccard相似性已经被广泛应用于各种类型的问题,其同样适用于分析复杂网络^[12]。它是一种用于衡量两个集合之间相似度的指标,定义为两个集合交集的大小与它们并集的大小之比。在大型软件系统中,代码重用是常见的现象,Jaccard相似性可以帮助识别相似或重复的代码段,这些代码段可能会共享相同的漏洞。

针对图中未充分利用边的传递信息能力的问题提出了一种利用边权重进行漏洞检测的方法。本文的工作如下:

1) 本文提出了一种基于边权重的软件漏洞检测方法(Edge Weight for Vulnerability Detection, EWVD),通过无加权网络的连接结构、邻居节点的重要性和Jaccard相似性来计算节点之间的边的重要性,这三个度量统称边权重。该方法在利用与漏洞语句相关的节点基础上,结合节点之间的边权重来提高漏洞检测性能。

2) 从SARD和NVD两个漏洞数据库中筛选了大量函数级的C/C++源代码样本,设计实验将本文方法与七种漏洞检测方法进行对比。与现有方法主要集中于节点特征不同,本文从边的角度出发,为无加权图构建边的综合权重度量方法,细化图中边的权重分配,从而引导模型关注对漏洞检测更具价值的潜在边信息。实验结果表明,EWVD优于目前先进的基线模型。

论文的其余部分组织如下:第1节回顾了相关工作,第2节详细介绍EWVD的设计,第3节给出了实验评价和分析,第4节对本文方

法的有效性进行分析。最后,第5节对本文进行了总结以及计划的未来工作。

1 相关工作

早期的软件漏洞检测研究主要使用基于静态规则和基于代码相似度匹配的方法。

基于静态规则的方法通过使用大量精心定义的漏洞规则或模式扫描目标源代码来进行。典型的静态分析器,如使用RATS^[2]、Checkmarx^[3]和Flawfinder^[4]都属于这一类。然而,用于扫描代码的规则主要由经验丰富的安全专家总结,每个规则都描述了一个特定的漏洞场景。因此,正如现有研究所证实的那样,这一类方法仅对某些类型的漏洞有效(例如缓冲区溢出、释放后使用),并且由于现实世界代码的多样性和可能的漏洞模式,在检测漏洞时往往会存在很高的FPR和FNR。

基于代码相似度匹配的方法通常计算目标代码的抽象表示与已知漏洞代码之间的相似性,以确定目标代码中是否存在潜在漏洞。现有相似性匹配方法采用的具体代码表示各不相同,例如Kim等人^[5]设计了一种安全感知抽象技术,使VUDDY对克隆代码中的常见修改具有弹性,同时即使在应用抽象后也能保留易受攻击的情况。Yamaguchi等人^[6]的方法通过从代码中提取抽象语法树并确定这些树中的结构模式来进行,这样代码中的每个函数都可以被描述为这些模式的混合,这种表示方式能够分解已知的漏洞并将其推断结果放入代码库中。Pham等人^[7]提出的SecureSync是使用先前已知漏洞的半自动构建的知识库,包括相应的系统、库以及易受攻击和修补的代码,可以自动检测共享源代码或库的不同系统中重复出现的软件漏洞。总体而言,它们擅长检测由代码或库重用引入的重复漏洞,而不是一般或未公开的漏洞。

随着人们发现深度学习方法在挖掘深层特征上更加具有优势,一些研究者开始尝试将其引入源代码漏洞检测任务。同时,为适应不同表征形式的代码数据,用于漏洞检测任务的深度学习模型在模型选择和设计上也有明显的差别。当前的方法主要通过序列和图对代码进行表征。

由于深度学习在自然语言处理领域的成功,研究者们便把代码视为自然语言序列,将代码通过序列的形式表征,应用于各种源代码相关的任务上,例如Li等人^[13]提出了一种基于程序切片的方法VulDeePecker,该方法基于库/API函数调用对源代码进行切片,并将切片后的代码放入RNN中检测漏洞。Ren等人^[14]改进了一种自动提示调节技术,其通过使用预训练模型的输出作为预测输出,重新表述任务输入并添加提示来模拟预训练模型的训练过程。同时引入强化学习奖励机制,旨在鼓励模型学习如何最大化奖励或最小化惩罚,从而提高性能。李妍等人^[15]优化了现有的程序切片方法,结合过程内和过程间切片对跨函数的漏洞进行上下文分析以捕获漏洞,应用Transformer神经网络模型关注来自不同表示子空间的信息来提取节点的深层特征,相较于循环神经网络解决了信息衰减的问题。

代码不同于普通的文本数据,其拥有局部的文本共现特征的同时,也具有丰富的结构信息。于是,为了弥补基于序列的代码表征方式在源代码漏洞检测任务中的不足,研究者们又提出基于图的漏洞检测方法。Tang等人^[16]设计了一种残差图注意力网络模块,在该模块中提出了一种均值双仿射注意力池机制来学习和更新沿控制流方向的节点特征,可以更有效地提取全局的图漏洞特征。胡雨涛等人^[17]对四种漏洞关注点进行切片,利用GGNN提取代码属性图(Code Property Graph, CPG)的特征检测漏洞,并对漏洞检测模型输出的检测结果提供了细粒度的解释。基于深度学习的漏洞检测方法已取得显著进展,从最早的序列模型,再到图神经网络处理代码结构特征,研究者们不断优化漏洞检测的精度和效率。

这些基于图的漏洞检测方法在多个与源代码相关的任务上表现出色,此外,部分研究进一步引入有向加权图的表征形式。其中,Cui等人^[18]提出了一种基于加权特征图的方法VulDetector。值得注意的是,该方法中的加权并非体现在图的边上,而是通过为每个节点分配

不同的权重来度量对目标语句的敏感程度。具体而言, VulDetector首先精确定位对漏洞敏感的关键字,以对函数的控制流图进行切片,从而在不损害安全相关语义的情况下降低图的复杂度。其次, VulDetector将源代码的抽象语法树表达式嵌入到一个语法特征向量中,并将其分配给每个节点。因此,加权特征图可以在不同程度的安全性下表征代码特征。另外, Liu等人^[19]采用滑动窗口组合方法,以代码语句为节点,连接滑动窗口内共现的代码语句,将共现的次数作为权重赋值给代码图中的边。然而,该方法只是考虑了共现次数这一个因素,若假设滑动窗口的大小为 n ,则除前 $n-2$ 个和倒数 $n-2$ 个节点外,其余的所有相邻节点之间都会存在一个权重相同且为 $n-1$ 的边,导致边的权重分布过于均衡,难以有效区分边在图结构中的相对重要性。

针对上述问题,本文提出一种新的有向加权图表示方法,旨在通过引入边权重的差异性来有效捕获程序结构和边潜在信息,具体而言,在构建代码图结构的过程中,通过融合连接结构、邻居节点的重要性和Jaccard相似性三个度量,对图中边的权重进行细粒度的刻画,这有助于弥补现有方法中对边权重方面粗糙表示的不足。

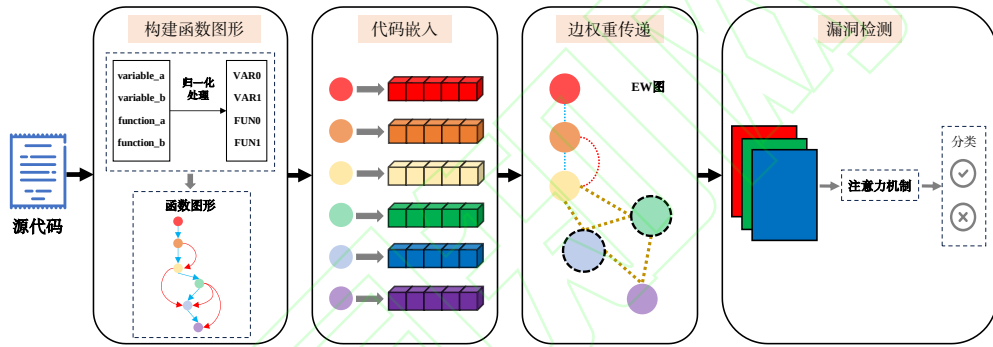


图1 EWVD方法框架

Fig. 1 The framework of EWVD

2.1 构建函数图形

EWVD是一种函数级的漏洞代码检测方法,本文将一段代码对应于一个图。因此,这个阶段的目标是生成一个程序依赖图(Program Dependency Graph, PDG)。由于源代码中

有注释、自定义变量名和函数名,在解析代码之前执行以下步骤。

步骤1:由于源代码中的注释会产生噪声,因此执行删除操作。

步骤2:将自定义的个性化变量名更改成规范变量名,即VAR1, VAR2……,例如(len = Command_Injection(data) → VAR = Command_Injection(VAR2))。

步骤3:将自定义的个性化函数名更改成规范函数名,即FUN1, FUN2……,例如(VAR1 = Command_Injection(VAR2) → VAR1 = FUN1(VAR2))。

经过上述三个步骤,实现了源代码的清理和抽象表示。接下来,使用Joern^[21]解析C/C++函数。Joern可以解析单个函数并分析函数的语义关系,它通常标识数据依赖关系和控制依赖关系。最后,得到了单个函数的程序依赖图,程序依赖图的节点对代码中的语句做出响应,其边包含数据流和控制流。如图2中的程序依赖图所示,这是一个缓冲区溢出的函数,将会用于以下方法的漏洞举例,其中蓝色边为控制流,红色边为数据流。

2.2 代码嵌入

现有的神经网络模型只能处理数值输入,这需要将符号代码转换为数值向量。一般来说,图节点有相应的节点嵌入方法,例如, Struc2vec^[22]、Node2vec^[23]。对于这项任务,本文选择了Sent2vec^[24]进行向量表示,因为在[25]中计算了Struc2vec、Node2vec和Sent2vec三种方法的质心距离,其表明质心距离越大,漏洞样本和非漏洞样本两

2 本文方法

多数现有的图表示方法忽略了网络中的边,从而难以捕捉边中隐藏的代码信息。因此,本文对Nandi等人^[10]提出的应用于病毒传播中的边权重度量进行改进,识别有传递信息能力的代码行进行漏洞检测。如图1所示,EWVD包括四个主要阶段:构建函数图形、代码嵌入、边权重传递以及漏洞检测。

1)构建函数图形:从初始程序文件中提取函数级样本,这些样本经过归一化处理。这个过程包括三个主要步骤:删除注释、标准化函数名和标准化变量名,处理之后解析源代码形成图形。

2)代码嵌入:由于网络中的每个节点都表示函数中的一行代码。实验中将一行代码看作一个句子,并将它们嵌入到一个向量中。

3)边权重传递:通过网络的连接结构、邻居节点的重要性和节点间的Jaccard相似性来计算两个相邻节点的边权重,识别有影响力的节点。

4)漏洞检测:将具有影响力的传递节点结合VulCNN^[20]中的图像生成技术,用于卷积神经网络(Convolutional Neural Network, CNN)模型的后续训练和分类,并结合注意力机制调整每个特征图的重要性,实现漏洞检测。

种类型之间的分离越好。Struc2vec、Node2vec和Sent2vec的质心距离分别为0.2768、0.3066和0.3371。Sent2vec的质心距离最大,表明类型分离度大,三种嵌入方法的漏洞检测性能对比见3.5节的实验。Sent2vec是一种无监督方法,可以在没有大量标注数据的情况下学习代码的表示,并且能够将整个代码语句编码成一个固定长度的向量,这个向量可以捕捉代码中潜藏的语义信息和逻辑结构,而不仅仅是表面的语法。相比单词向量,语句向量更适合处理代码语言中与句子级别相关的任务。例如,语句向量可以捕捉到一个函数在逻辑上是否与已知的存在漏洞的函数相似,而单词向量仅仅关注单个代码符号或关键词的语义信息,这会导致忽略代码的全局逻辑或功能。

2.3 边权重传递

在PDG中,漏洞的传播往往受到代码结构和信息流的影响,因此从连接结构、邻居节点的重要性和Jaccard相似性三个边权重度量来衡量边权重的作用。

2.3.1 连接结构

节点(即代码行)之间的连接表明它们有形成三角形结构的趋势,一旦形成三角形,这三个节点就会相互交换信息^[11]。显然,节点邻居的三角形结构的比例在信息传递中起着重要作用。在信息传递过程中,两个相邻的节点共享同一个邻居,它们将有更大的概率相互传递信息^[11]。因此,节点与其邻居形成的三角形结构越多,它们就越有可能相互传递是否含有漏洞的信息。基于这一概念,通过公式(1)计算节点 v_A 的三角形结构 $TP(v_A)$ 的百分比:

$$TP(v_A) = \frac{NTC(v_A)}{TC} \quad (1)$$

其中, $NTC(v_A)$ 为节点 v_A 与其相邻节点的三角形结构个数, TC 为网络中三角形结构的总数, TC 的计算方法是依据networkx库中的triangles

函数。基于图2的程序依赖图的变化,三角形结构如图3所示,图中

共有4个三角形连接结构,

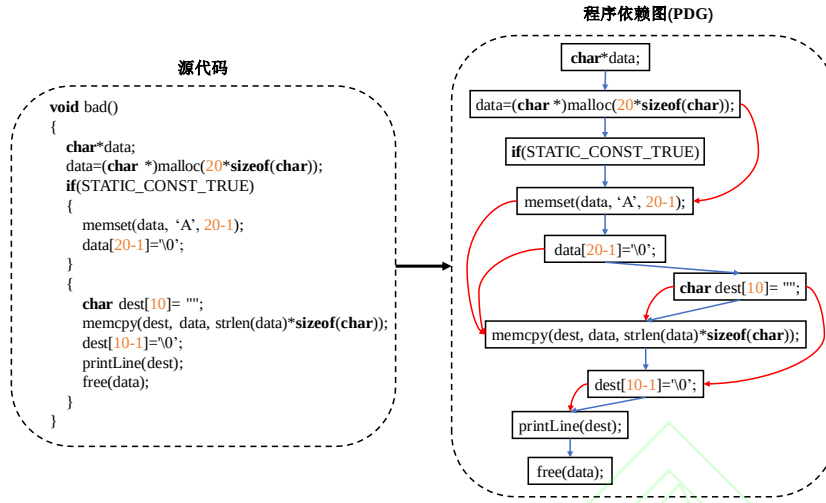


图2 程序依赖图(PDG)

Fig. 2 The program dependency graph (PDG)

分别用虚线连接。三角形结构之所以有更高的概率进行信息交换,是因为存在额外的路径促进信息传递。如果有多个路径连接节点,那么即使其中一条路径失效,信息仍然可以通过其他路径传递。

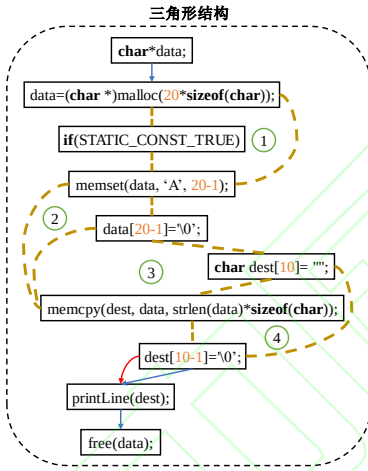


图3 三角形连接结构

Fig. 3 The triangle connectivity structure

2.3.2 邻居节点的重要性

当一个节点连接到更重要的节点时,该节点的重要性潜力就会增加。因此,节点的传递能力不仅取决于其自身的属性,还取决于它所连接的邻居节点的重要性。在程序依赖图中使用度中心性(Degree Centrality, DC)^[26]考虑邻居节点的局部重要性,度中心性反映了节点直接连接的邻居数量。同时,接近中心性(Closeness Centrality, CC)^[26]分析邻居节点的全局重要性,它衡量的是一个节点通过其邻居节点在整个网络中的影响力。度中心性和接近中心性的数值越高,说明该节点在网络中的连通性越好,即代码行在函数中的重要性越强。节点 v_A 邻居节点的重要性通过公式(2)进行计算。

$$NIP(v_A) = \sum_{v_a \in \eta(v_A)} \sqrt{DC(v_a) * CC(v_a)} \quad (2)$$

其中, $\eta(v_A)$ 为 v_A 的邻居节点,用 v_a 表示。通过度中心性,可以迅速定位程序依赖图中直接连接较多的节点。这些节点通常具有更高的影响力和信息传递能力,是漏洞传播的主要途径。接近中心性可以识别那些虽然直接连接较少但在全局范围内具有高度影响力的节点。这些节点是信息传递的核心桥梁,即使它们的度中心性较低,但在漏洞传播路径中起着至关重要的作用。如图4所示,该图的示例是图2

中的函数,代码每一行的数值都由公式(2)计算得出,颜色随着值的增加而变深,深红色表示节点最重要。度中心性高的节点往往是漏洞传播的起点或中转站,而接近中心性高的节点可能通过其邻居影响更广泛的区域,两者结合可以帮助漏洞检测系统在局部和全局层面上找到最有可能成为漏洞传播渠道的途径。

void bad() {	DC	CC	NIP
char*data;	1/9	9/30	0.37
data=(char *)malloc(20*sizeof(char));	3/9	9/22	0.96
if(STATIC_CONST_TRUE)	2/9	9/23	0.85
{memset(data, 'A', 20-1);	4/9	9/17	1.62
data[20-1]='\0';	3/9	9/19	1.49
char dest[10]= "";	4/9	9/20	1.41
memcpy(dest, data, strlen(data)*sizeof(char));	5/9	9/16	1.79
dest[10-1]='\0';	4/9	9/19	1.35
printLine(dest);	3/9	9/25	0.63
free(data);}	1/9	9/33	0.35

图4 邻居节点的重要性计算图

Fig. 4 The calculation graph of the importance of neighboring nodes

2.3.3 Jaccard相似性

如果两个代码节点有很多共同的邻居(即它们的Jaccard相似性较高),则可以推测这些节点之间的联系很强。Jaccard相似性可以通过其用于衡量两个节点之间的相似度来计算。具体来说,Jaccard相似性衡量的是两个节点的共同邻居数量占它们全部邻居数量的比例,如公式(3)所示。

$$Jaccard(v_A, v_B) = \frac{N(v_A) \cap N(v_B)}{N(v_A) \cup N(v_B)} \quad (3)$$

其中, $N(v_A)$ 和 $N(v_B)$ 分别表示两个节点 v_A 和 v_B 的邻居集合($v_A \neq v_B$)。以图5中的节点“memset (data, 'A', 20-1);”和“data[20-1]='\0';”为例,它们之间的公共邻居只有一个,即“memcpy (dest, data, strlen (data) * sizeof (char));”,结合邻居个数可得出两个节点间的Jaccard相似性为1/7。Jaccard相似性意味着这些节点处于同一个依赖或逻辑区域,且很可能在漏洞传播路径上。Jaccard相似性高的节点表示它们有更大概率相互影响,因此如果一个节点已经发现了漏洞,与它相似性高的节点应特别关注。如果一个无漏洞节点与漏洞节点的相似性较高,则它有更大可能性受到漏洞影响,这些节点同样需要引起重视。

这三种度量方式分别从不同的角度刻画漏洞的传播特征。首先,连接结构通过三角形结构的比例来衡量节点信息交换的潜力,三

角形结构的存在意味着多个路径可用于信息传递,使得漏洞信息即使在某条路径受阻时仍可通过其他路径传播。其次,邻居节点的重要性中DC高的节点连接了更多的邻居,通常承担着关键的数据或控制流作用,是漏洞传播的主要通道;而CC高的节点虽然直接邻居较少,但在整个PDG中具有较短的平均路径长度,即处于PDG的核心位置,导致漏洞影响更远的代码区域。最后,Jaccard相似性用于判断不同的节点即代码语句是否“经常操作同一批变量”或者“被同一类语句控制”,相似性高的节点通常执行相似的逻辑,因如果某个节点已知存在漏洞,与其具有高Jaccard相似性的节点也更能受到漏洞影响。

void bad() {	邻居个数
char*data;	1
data=(char *)malloc(20*sizeof(char));	3
if(STATIC_CONST_TRUE)	2
{memset(data, 'A', 20-1);	4
data[20-1]='\0';}	3
{char dest[10]="";	3
memcpy(dest, data, strlen(data)*sizeof(char));	4
dest[10-1]='\0';	3
println(dest);	2
free(data);}	1
}	

图5 函数中节点的邻居个数

Fig. 5 The number of neighbors of a node in a function

结构相关性主要由连接结构和 Jaccard 相似性衡量,其中三角形结构反映了信息在局部区域的流动能力,而 Jaccard 相似性的高相似度表示这些代码在执行逻辑上紧密关联,因此漏洞可能沿着这些路径传播。邻居节点的重要性的节点之间通过DC和CC的度体现代码行之间的语义关联。如果一个代码行节点在PDG中既具备高结构相关性(共享大量路径),又具备高语义相关性,则该代码行更可能成为漏洞传播路径上的关键节点。

因此,连接结构、邻居节点的重要性和 Jaccard 相似性组成了边权重传递,算法1展示了节点的传递能力的计算过程。

算法1 边权重算法。

输入:图 $G = (V, E)$

输出:图 G 的 $TP(v_A), NIP(v_A), Jaccard(v_A, v_B)$

1) $TP(v_A), NIP(v_A), Jaccard(v_A, v_B) \rightarrow \{\}, \{\}, \{\}$

2) for each node $v_A \in V$ do

3) $TP(v_A) \leftarrow$ 节点 v_A 三角形结构的百分比

4) $TP(v_A).append(\{\})$

5) end for

6: for each node $v_A \in V$ do

7) $NIP(v_A) \leftarrow$ 节点 v_A 邻居节点的重要性

8) $NIP(v_A).append(\{\})$

9) end for

10) for each node $v_A \in V$ with $v_B \in v_A$ 的邻居节点 and edge $(v_A, v_B) \in E$ do

11) $Jaccard(v_A, v_B) \leftarrow$ 节点 v_A 与节点 v_B 之间的 Jaccard 相似性

12) end for

13) return $TP(v_A), NIP(v_A), Jaccard(v_A, v_B)$

首先构建三个新的空字典,以便保存不同的边权重。然后,根据以上所述的公式(1)、(2)、(3)分别对图中的所有节点和边进行边权重计算。最后,以节点和边为键,边权重为值构成键值对放入三个不同的空字典中。

2.4 漏洞检测

在计算出代码嵌入阶段之后,得到了一个具有节点向量的程序

依赖图。此阶段的目标是使用 ValCNN 中的图像生成技术将网络的连接结构、邻居节点的重要性以及 Jaccard 相似性乘以节点向量值放入图像的 RGB(Red-Green-Blue)通道,同时结合注意力机制关注不同通道的重要性。

网络是一个复杂的图形,程序依赖图也是一种图结构。可以从三个不同的方面(即连接结构、邻居节点的重要性以及 Jaccard 相似性)计算图形函数中所有代码语句的贡献程度。它们分别对应于图像的 RGB 通道,也就是说,连接结构对应于 R,邻居节点的重要性对应于 G, Jaccard 相似性对应于 B,通过这三个通道将生成一个图像,图6是代码图像生成过程。具体来说,可以通过方程式(1)、(2)、(3)得到的结果与节点向量值相乘,计算出的值放入到三个 RGB 通道。

这时,获得了一个 RGB 图像,这是将函数的源代码转换为图像表示。在漏洞检测领域,采用先进技术来处理源代码的复杂性和细微差别是必不可少的。CNN 凭借其辨别复杂特征和模式的能力已成为各种基于图像的任务中的强大模型,将图结构转换为图像的目的是利用 CNN 在图像领域的效率。因此,实验将利用 CNN 对图像的高效处理能力来减少源代码漏洞的检测时间。

本方法所采用的 CNN 使用的卷积核形状为 3×128 ,其中 3 决定了同时考虑的节点数。(即卷积核大小)的值范围为 1 到 10,每个卷积核生成 32 个特征图,对输入数据进行卷积操作,使用 Relu 激活函数增加非线性能力,通过最大池化操作提取最显著的特征,然后将不同卷积核的结果拼接在一起,再经过 Dropout 层防止过拟合,最终通过长度为 320 的全连接层将特征映射到最终的分类类别。EWVD 中使用的各项参数的详细描述,请参考表 1。

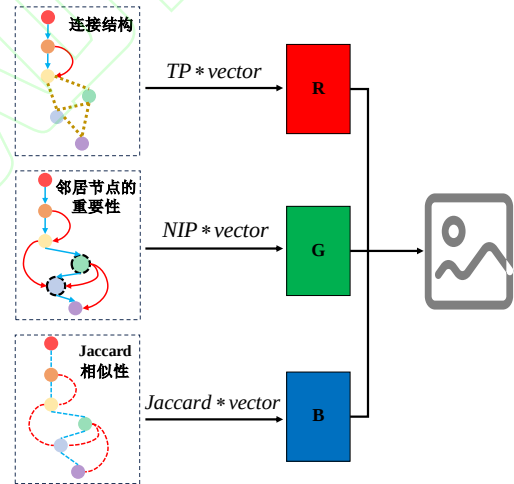


图6 代码图像生成过程

Fig. 6 The process of code image generation

由于 CNN 模型需要大小一致的输入图像,并且不同函数中的代码行数量可能会有所不同,因此选择合适的阈值长度进行扩展或裁剪变得至关重要。通过实验发现,当代码行阈值为 120 的时候,漏洞检测性能达到最佳状态。如图 7 所示,代码行阈值与 Accuracy 呈正相关。阈值越大,Accuracy 越高,直到在 120 处,Accuracy 达到稳定状态。对于代码长度小于 120 的函数,在末尾用零填充向量。如果函数超过 120 行,将截断向量的尾部。输入图像大小设置为 $3 \times 120 \times 128$, 3 代表三个通道(连接结构、邻居节点的重要性和 Jaccard 相似性),120 是代码行阈值,128 表示句子向量的维度。

在 CNN 模型后引入多头注意力机制,使其能够关注图像中不同通道的重要性。每个输入通道的特征通过独立的多头自注意力层进行处理,允许模型在不同的上下文中动态地关注输入序列的不同部分。将每个通道的数据特征分成 4 个不同的子空间,每个子空间独立计算其注意力权重,将状态向量矩阵聚合在隐藏层中增强模型的重要特征。最后,将处理后的特征与卷积层的输出结合,从而生成更丰富的特征表示,神经网络结构如图 8 所示。

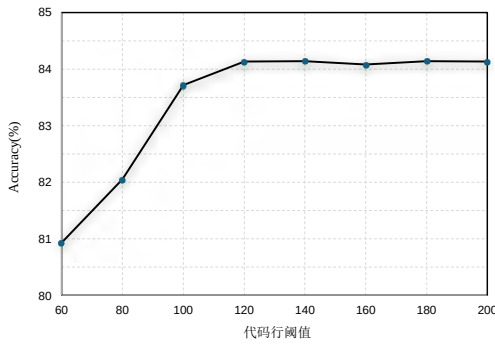


图7 不同的代码行阈值对EWVD的性能影响

Fig. 7 The impact of different code line thresholds on the performance of EWVD

3 实验

本文将数据集按照9:1的比例划分为训练集和测试集,采用十折交叉验证来验证本文方法的有效性。模型的超参数设置如表1所示。

实验中使用的CPU型号为Intel(R) Xeon(R) Platinum 8352V CPU @ 2.10GHz, GPU型号为NVIDIA RTX 3090, RAM为64G, 操作系统为Ubuntu 22.04, Python版本为3.10.13, Pytorch版本为2.1.1, Networkx版本为3.1, Joern版本为1.1.1113。

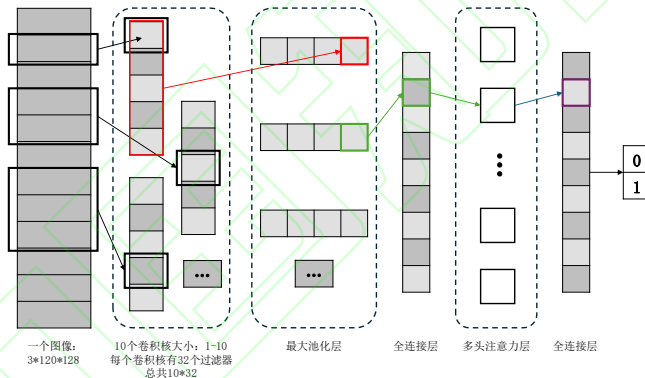


图8 EWVD的神经网络结构

Fig. 8 The neural network structure of EWVD

据,包含来自几个开源项目的非漏洞函数。SARD是一个被广泛用作生成实验样本的漏洞数据库,它的程序是生产代码、学术代码和合成代码的混合,每个程序都与“坏”、“好”或“混合”标签相关联。每个“坏”程序通常包含一个漏洞函数,而每个“好”程序只包含从其漏洞版本中修复的非漏洞函数,每个“混合”程序都将漏洞函数及其补丁版本合并在一个程序中。具体数据信息如表

2所示。

表2 数据集集中的样本数

Tab. 2 The number of samples in the dataset

数据集	漏洞函数	非漏洞函数
NVD	1384	5913
SARD	12303	21057

最终合成的数据集共有40657个函数样本,其中的漏洞函数样本占比33.7%(13687个),非漏洞函数样本占比66.3%(26970个)。

3.3 评价指标

漏洞检测是一个二分类问题,因此本文使用了常用的分类指标: Accuracy、FPR和FNR。Accuracy表示正确预测样本的比例, FPR表示在所有实际没有漏洞的样本中被错误检测为有漏洞的比例, FNR表示在所有实际有漏洞的样本中被错误检测为没有漏洞的比例。

表1 模型中的参数设置

Tab. 1 The parameter settings in the model

参数	设置
损失函数	交叉熵损失函数
激活函数	ReLU
优化器	Adam
batch大小	32
学习率	0.001
迭代次数	200

3.1 研究问题

为了验证本文方法的有效性,本文提出了三个研究问题:

RQ1:EWVD的漏洞检测性能如何?

RQ2:连接结构、邻居节点的重要性和Jaccard相似性三者是否有助于EWVD检测漏洞?

RQ3:EWVD的运行时间开销相比于基线方法如何?

3.2 数据集

为了评估实验中的方法,本文使用的是VulCNN中的数据集,这个数据集是在国家漏洞数据库(National Vulnerability Database, NVD)和软件保障参考数据集(Software Assurance Reference Dataset, SARD)的基础上构建了一个由C/C++函数组成的数据集。NVD是由美国维护的一个漏洞数据库,它包含了大量真实世界的程序漏洞。实验选取的漏洞函数,这些程序属于用C/C++编写的不同开源软件,还随机选择[27]中数据集的一部分作为真实世界的数

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (4)$$

$$FPR = \frac{FP}{FP + TN} \quad (5)$$

$$FNR = \frac{FN}{FN + TP} \quad (6)$$

其中, TP (True Positive)表示模型将真实存在的漏洞正确标记为漏洞的数量, FP (False Positive)表示模型将实际上不存在的漏洞误标记为漏洞的数量, TN (True Negative)表示模型正确识别出没有漏洞的代码并未标记为漏洞的数量, FN (False Negative)表示模型没有检测出真实存在的漏洞。

3.4 基线方法

为了评估本文方法的有效性,将EWVD与七种漏洞检测方法相比较,包括两种传统方法RATS^[2]和FlawFinder^[4],以及五种基于深度学习的方法,即VulDeePecker^[13]、VGDetecter^[28]、VulDeeLocator^[29]、Deign^[30]和VulCNN^[20]。

1)RATS:对源代码进行词法分析,将代码分解为tokens,通过正则表达式和字符串匹配技术,检测代码中与漏洞模式匹配的部分。

2)FlawFinder:维护一个包含危险函数和漏洞模式的数据库,通过简单的语法分析识别函数调用和参数传递,每个函数都有一个风

险评分(从0到5),用于表示其潜在的危险程度。

3) VulDeePecker:首先根据代码片段中的“关键点”(例如库/API函数调用)对输入代码片段进行切片,然后使用LSTM对切片的代码片段进行编码并进行预测。

4) VGDetecter:首先对源代码进行程序分析,提取与控制流相关的语义特征,然后将其与图卷积网络相结合,将代码片段嵌入潜在特征空间。

5) VulDeeLocator:将源代码看作是由多个“token”(词或符号)组成的序列,然后使用双向循环神经网络检测漏洞。

6) Devign:通过AST、CFG、DFG和代码执行顺序组合的联合图来表示源代码,并通过连接的边明确指出代码执行流程和数据传递流程。同时采用门控图神经网络学习联合图的代码表示,进一步进行源代码漏洞检测。

7) VulCNN:将源代码转换为图像,并将这些图像输入到卷积神经网络(CNN)中进行训练,在扫描大规模源代码漏洞时实现可扩展性和准确性。

RATS和FlawFinder依赖于专家定义的规则或模式,VulDeePecker和VulDeeLocator都采用切片方法处理数据集,VGDetecter提取代码的控制流,Devign和VulCNN则是分别提取代码的CPG和PDG。

3.5 实验结果

针对RQ1,本文将EWVD与七种漏洞检测方法(RATS、FlawFinder、VulDeePecker、VGDetecter、VulDeeLocator、Devign和VulCNN)进行比较,性能结果如表3所示。EWVD在Accuracy、FPR和FNR方面分别达到84.17%、17.16%和16.43%。

表3 七种基线方法和EWVD的漏洞检测结果

Tab. 3 The vulnerability detection results of seven baseline methods and EWVD

方法	FPR(%)	FNR(%)	Accuracy(%)
RATS	27.91	64.43	59.17
FlawFinder	54.67	47.62	59.08
VulDeePecker	25.37	14.95	76.68
VGDetecter	26.14	16.62	78.53
VulDeeLocator	21.71	14.36	80.72
Devign	24.76	13.37	83.06
VulCNN	18.27	17.47	83.11
EWVD	17.16	16.43	84.17

表4 嵌入方法的漏洞检测性能对比

Tab. 4 The comparison of vulnerability detection performance of embedding methods

嵌入方法	FPR(%)	FNR(%)	Accuracy(%)
Struc2vec	29.26	29.10	72.69
Node2vec	27.15	26.28	74.52
Sent2vec	17.16	16.43	84.17

从表3可以看出,EWVD在Accuracy和FPR指标中是最好的。相较于性能最好的基线方法,EWVD的Accuracy提高1.06个百分点,FPR降低1.11个百分点。两个传统方法RATS和FlawFinder的Accuracy值都表现较差,这归因于它们依赖于简单的解析方法和人为定义的漏洞规则,使得检测漏洞的能力受到阻碍,导致Accuracy很低。对于VulDeePecker来说,虽然它采用程序切片来处理数据集,但忽略了代码中的关键依赖信息,因此它的性能最差。由于VGDetecter只考虑了代码之间的控制依赖性,而没有数据依赖性,从而导致高FPR。VulDeeLocator利用LLVM(Low Level Virtual Machine)中间表示(Intermediate Representation, IR)容纳更多的程序语义,它相比VulDeePecker包含更多的代码信息。然而,同样由于切片的原因,导致VulDeeLocator的性能低于EWVD。Devign采用CPG来表示代码,然后使用图神经网络检测漏洞,这导致其FPR较高。虽然VulCNN的FNR不是最低的,但其在FPR和Accuracy方面的表现使其成为所选

方法中最优的基线方法,它通过中心性分析关注节点(即程序语句)的中心性度量,在此基础上将源代码转换成图像进行漏洞检测,但是该方法只关注了网络中的节点,而忽略了潜在的边信息,所以VulCNN的性能比EWVD较低。

为直观展示三个嵌入方法在具体指标上的差异,表4给出了详细的性能对比结果。从表4中可以看出Sent2vec性能更优越,与Node2vec相比,Sent2vec的准确率(Accuracy)提高9.65个百分点,FPR和FNR分别降低9.99个百分点和9.85个百分点。

实验基于十折交叉验证的结果进行配对t检验,检验结果如表5和表6所示,EWVD的Accuracy($0.01 \leq p < 0.05$)和FPR($p < 0.01$)相比VulCNN具有显著提升。EWVD相比与一些基线方法FNR的t统计量为正,说明在某些情况下EWVD存在较高的漏报现象。t统计量是两个配对样本的均值差异相对于样本自身波动的大小,t统计量越远离0,说明差异越大。p值是评估统计学中检验显著性的度量指标,表明其Accuracy的提升和FPR的降低不是随机偶然的,而是具有统计学意义的改进。

回答RQ1:与基于深度学习的方法相比,EWVD通过边权重充分利用了边信息,实现了更优的检测性能。

表5 t统计量显著性分析

Tab. 5 The statistical significance analysis of t-statistic

方法	FPR	FNR	Accuracy
RATS	-93.39	-228.26	164.25
FlawFinder	-195.85	-253.15	141.46
VulDeePecker	-44.46	8.03	41.81
VGDetecter	-61.94	0.58	44.40
VulDeeLocator	-19.08	15.85	17.28
Devign	-44.79	20.16	5.58
VulCNN	-7.40	-4.47	3.169

表6 p值显著性分析

Tab. 6 The statistical significance analysis of p-value

方法	FPR	FNR	Accuracy
RATS	9.38×10^{-15}	3.03×10^{-18}	5.85×10^{-17}
FlawFinder	1.20×10^{-17}	1.19×10^{-18}	2.24×10^{-16}
VulDeePecker	7.37×10^{-12}	2.15×10^{-5}	1.28×10^{-11}
VGDetecter	3.76×10^{-13}	0.576	7.45×10^{-12}
VulDeeLocator	1.37×10^{-8}	6.98×10^{-8}	3.29×10^{-8}
Devign	6.89×10^{-12}	8.46×10^{-9}	0.000342
VulCNN	0.000413	0.00155	0.0114

针对RQ2,本文分别对连接结构、邻居节点的重要性和Jaccard相似性三个特征进行消融实验,评估它们对漏洞检测性能的贡献。消融方式包括:完全去除所有特征EWVD_{无EW},仅保留单独一个特征EWVD_{TP}、EWVD_{NIP}和EWVD_{Jac},保留两个特征EWVD_{TP,NIP}、EWVD_{TP,Jac}和EWVD_{NIP,Jac}。

如表7所示,在完全去除连接结构、邻居节点的重要性和Jaccard相似性特征后,模型的漏洞检测性能显著下降,表现在关键节点识别和漏洞传播路径预测的准确性上都表现不佳,且FPR和FNR也随之上升。当仅保留单个特征时,模型能够捕捉到节点之间的基本结构关系,对漏洞传播路径有一定识别能力,但在准确性上有所欠缺。当保留两个特征时,模型能够在局部和全局结构上识别出重要的节点和传播路径,性能较单个特征差别不大。

三个特征相结合,使得模型在检测漏洞时能够从多个角度评估节点之间的信息传播能力。连接结构提供了连接结构层面的支持,邻居节点的重要性聚焦于关键节点的间接影响,而Jaccard相似性则关注代码之间的相似性。因此,将这三个特征集成在模型中,可以有效增强其在复杂依赖关系中的漏洞检测效果,确保模型能够准确判断潜在漏洞传播路径的权重,并识别关键的高风险节点。

回答RQ2:连接结构、邻居节点的重要性和Jaccard相似性三者对模型有显著影响,尤其是结合使用时,能够提升漏洞检测的性能。这

表明将这三个特征集成到模型中可以更全面地捕捉漏洞传播的潜在路径和高危漏洞。

表7 消融实验

Tab. 7 The ablation experiment

消融方式	FPR(%)	FNR(%)	Accuracy(%)
EWVD _{wu_EW}	19.47	17.46	82.67
EWVD _{TP}	17.49	16.51	83.59
EWVD _{NIP}	17.71	16.64	83.59
EWVD _{Jac}	17.26	16.63	83.61
EWVD _{TP_NIP}	17.34	16.80	83.85
EWVD _{TP_Jac}	17.49	16.49	83.76
EWVD _{NIP_Jac}	17.67	16.76	83.60
EWVD	17.16	16.43	84.17

针对RQ3,本文对EWVD的运行时间开销进行评估。给定一个函数,EWVD通过构建函数图形、代码嵌入、边权重传递、漏洞检测这四个步骤来完成分类。这四个步骤对应的平均运行时间开销分别是1.69秒、0.00047秒、0.5秒和0.000059秒,所以EWVD对一个函数进行检测的平均运行时间开销约为2.2秒。实验排除了RATS和FlawFinder的比较,因为它们不是基于深度学习的方法,而且比其他五种基于深度学习的方法表现更差。图9展示了EWVD以及基于深度学习方法的平均运行时间开销。

从图9的运行时间开销中可以看出,VulDeePecker的切片生成阶段需要提取多种图,包括控制流图、程序依赖图和函数调用图,导致运行时间长于EWVD。VGDetecter中的图卷积网络模型在图规模较大且连接密集的情况下,运行时间开销不占优势。VulDeeLocator是一个基于LLVM-IR的漏洞检测系统,它需要先编译源代码,这会导致编译阶段消耗大量时间。Design与VulDeePecker的情况类似,它需要组合不同的代码表示,即抽象语法树、控制流图、数据流图和自然代码序列来实现准确的漏洞检测。EWVD虽然在边权重传递阶段消耗了一些时间,但是与VulCNN相比,两者的时间开销差距并不大。

回答RQ3:与其他基线方法相比,EWVD的运行时间开销较少,比运行时间最短的VulCNN只多了0.3秒。

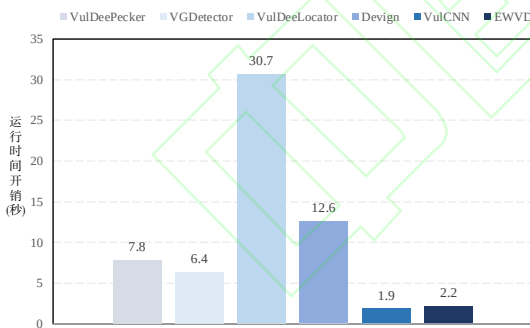


图9 基于深度学习的基线方法与EWVD的平均运行时间开销

Fig. 9 The average runtime overhead of deep learning-based baseline methods and EWVD

4 有效性分析

1)内部有效性

超参数和边权重是影响内部有效性的主要因素。EWVD由多个超参数控制,包括学习率、优化器、batch大小等。这些参数设置不同会产生不同的结果。然而,由于参数数量众多,探索最佳参数设置可能很困难。本文的研究目的不是寻求最佳参数设置,而是通过与基线方法的比较来证明本文方法的性能。因此,可以通过参数优化进一步提高性能。

如图10是一个CWE-119类型的漏洞示例,即对内存缓冲区边界的不恰当限制。这里的%s会将字符串从readLine中读取到readData,但readData没有设置长度限制。若readLine来自恶意数据

包或被人构造得足够长,会导致readData缓冲区溢出,从而触发CWE-119类型漏洞。VulCNN更依赖节点的全局中心性度量,它根据中心性度量分析代码行的度,而在while循环中的度(即控制流和数据流)是整个函数中最多的,这会影响VulCNN判断这一部分为漏洞点,但实际上并不是。EWVD通过数据流将141行的if语句、145行的wth→data_offset += strlen(readLine)和149行的if语句组成三角形连接结构,将三者视为信息交换节点,不同于VulCNN局限在while循环中。149行的if语句被包含在多个节点的Jaccard相似性的共同邻居中,同时它的邻居节点的重要性较高,因此149行的if语句被EWVD视为最有可能的漏洞区域。

2)外部有效性

数据集是影响外部有效性的主要因素。现阶段真实的漏洞数据集严重缺乏,SARD中的标记函数来自合成程序,可能无法代表整个程序。为了解决这个问题,实验中将NVD中的一些真实漏洞和开源产品中的样本函数添加到数据集中。此外,目前大部分数据集主要以独立函数的粒度进行标记。实际上,漏洞可能发生在跨函数的地方。因此,仅仅因为某个函数中存在漏洞而将其标记为易受攻击并不总是正确的,同时还应该考虑其他过程间信息,例如宏及其调用的函数,以形成完整的漏洞触发上下文。构建这种由精确标记的漏洞触发代码上下文组成的大规模数据集并不容易,这需要领域专家持续的努力。

```

129 /* Read the capture file sequentially
130 * Firespark scans the file with sequential reads during preview and initial display. */
131 static boolean
132 daintree_sma_read(wtap *wth, int *err, gchar **err_info, guint64 *data_offset)
133 {
134     guint64 seconds;
135
136     *data_offset = wth->data_offset;
137
138     /* we've only seen file header lines starting with 'd', but
139     * if others appear in the file, they are tossed */
140     do {
141         if (file_gets(readLine, DAINTREE_MAX_LINE_SIZE, wth->fh) == NULL) {
142             *err = file_error(wth->fh);
143             return FALSE; /* all done */
144         }
145         wth->data_offset += strlen(readLine);
146     } while (readLine[0] == COMMENT_LINE);
147
148     /* parse one line of capture data */
149     if (sscanf(readLine, "%s %G_GINT64_MODIFIER %u %u %s",
150             &seconds, &wth->phdr.ts.nsecs,
151             &wth->phdr.len, readData) != 4) {
152         *err = WTAP_ERR_BAD_RECORD;
153         *err_info = g_strdup("daintree_sma: invalid read record");
154         return FALSE;
155     }
156
157     wth->phdr.ts.secs = (time_t) seconds;
158     wth->phdr.ts.nsecs *= 1000; /* convert ms to ns */

```

图10 CWE-119漏洞示例

Fig. 10 The vulnerability example of CWE-119

5 总结与展望

本文提出了一种基于边权重的软件漏洞检测方法。该方法对源代码进行预处理,以增加具有相同语义的代码之间的相似性,并利用Joern工具提取PDG作为代码表示,以获得更丰富的语义信息。实验使用Sent2vec对PDG节点进行向量表示,并在复杂网络分析中增加对边信息的关注度,利用边权重度量分析节点的传递信息能力。最后,利用卷积神经网络将PDG转换成图像的技术进行特征提取,交由分类器进行分类。本文的主要贡献是通过无加权网络的连接结构、邻居节点的重要性和Jaccard相似性来计算节点之间的边的重要性,提高图表示中对边信息的利用来加强漏洞检测的能力。本文提出的方法在SARD和CVE数据集上的性能优于七种先进的漏洞检测方法。

本文方法仍然存在一些不足。首先,本文的方法主要是为检测C/C++程序源代码中的漏洞而设计的,计划在未来的工作中将该方法扩展到其他编程语言。其次,检测性能仍有改进的空间,本文只关注了边权重,而未考虑其他可能影响检测精度的因素,如图结构的复杂性或模型的训练策略等,这些因素也可能在一定程度上优化漏洞检测的效果。最后,由于大多数漏洞检测工具都是闭源的,实验中只将本文的方法与其他七种漏洞检测方法进行了比较。在未来的工作

中,计划对其他漏洞检测系统进行更全面的比较分析。

参考文献 (References)

- [1] STEENHOEK B, GAO H, LE W. Dataflow analysis-inspired deep learning for efficient vulnerability detection [C]// ICSE 2024: Proceedings of the 46th IEEE/ACM International Conference on Software Engineering. International Conference on Software Engineering. New York: ACM, 2024: 16:1-16:13.
- [2] Rough Audit Tool for Security [EB/OL]. [2024-11-07]. <https://code.google.com/archive/p/rough-auditing-tool-for-security/>.
- [3] Checkmarx [EB/OL]. [2024-11-07]. <https://www.checkmarx.com/>.
- [4] Flawfinder [EB/OL]. [2024-11-07]. <http://www.dwheeler.com/flawfinder/>.
- [5] KIM S, WOO S, LEE H, et al. VUDDY: A scalable approach for vulnerable code clone discovery [C]// S&P 2017: 2017 IEEE Symposium on Security and Privacy. Washington, DC: IEEE Computer Society, 2017: 595-614.
- [6] YAMAGUCHI F, LOTTMANN M, RIECK K. Generalized vulnerability extrapolation using abstract syntax trees [C]// ACSAC 2012: Proceedings of the 28th Annual Computer Security Applications Conference. New York: ACM, 2012: 359-368.
- [7] Pham N H, Nguyen T T, Nguyen H A, et al. Detection of recurring software vulnerabilities [C]// ASE 2010: Proceedings of the 25th IEEE/ACM International Conference on Automated Software Engineering. New York: ACM, 2010: 447-456.
- [8] 李韵, 黄辰林, 王中锋, 等. 基于机器学习的软件漏洞挖掘方法综述[J]. 软件学报, 2020, 31(7): 2040-2061. (Li Y, HUANG C L, WANG Z F, et al. Survey of software vulnerability mining methods based on machine learning[J]. Journal of Software, 2020, 31(7): 2040-2061.)
- [9] ALLAMANIS M, BROCKSCHMIDT M, KHADEMI M. Learning to represent programs with graphs [C/OL]// ICLR 2024: The Sixth International Conference on Learning Representations. (2018-02-16) [2024-11-07]. <https://openreview.net/forum?id=BJOFETxR->.
- [10] NANDI S, MALTA M C, MAJI G, et al. IS-PEW: Identifying influential spreaders using potential edge weight in complex networks [C]// COMPLEX NETWORKS 2023: The 12th International Conference on Complex Networks and their Applications. Switzerland: Springer Nature, 2023: 309-320.
- [11] MA X, MA Y. The local triangle structure centrality method to rank nodes in networks [J]. Complexity, 2019, 2019 (1): 9057194.
- [12] COSTA L F. Further generalizations of the Jaccard index [J/OL]. (2021-11-18) [2024-11-07]. <https://arxiv.org/abs/2110.09619>.
- [13] LI Z, ZOU D Q, XU S H, et al. VulDeePecker: A deep learning-based system for vulnerability detection [C/OL]// NDSS 2018: The 25th Network and Distributed System Security Symposium. (2018-1-5) [2024-11-07]. <https://arxiv.org/abs/1801.01681>.
- [14] REN Z, JU X, CHEN X, et al. ProRLearn: Boosting prompt tuning-based vulnerability detection by reinforcement learning[J]. Automated Software Engineering, 2024, 31(2): 38.
- [15] 李妍, 羌卫中, 李珍, 等. 基于程序过程间语义优化的深度学习漏洞检测方法[J]. 网络与信息安全学报, 2023, 9(6): 86-101. (Li Y, QIANG W Z, LI Z, et al. Deep learning vulnerability detection method based on optimized inter-procedural semantics of programs [J]. Chinese Journal of Network and Information Security, 2023, 9(6): 86-101.)
- [16] TANG M, TANG W, GUI Q, et al. A vulnerability detection algorithm based on residual graph attention networks for source code imbalance (RGAN) [J]. Expert Systems with Applications, 2024, 238: 122216.
- [17] 胡雨涛, 王溯远, 吴月明, 等. 基于图神经网络的切片级漏洞检测及解释方法[J]. 软件学报, 2023, 34(6): 2543-2561. (HU Y T, WANG S Y, WU Y M, et al. A slice-level vulnerability detection and interpretation method based on graph neural network [J]. Journal of Software, 2023, 34(6): 2543-2561.)
- [18] CUI L, HAO Z, JIAO Y, et al. VulDetector: Detecting vulnerabilities using weighted feature graph comparison [J]. IEEE Transactions on Information Forensics and Security, 2021, 16: 2004-2017.
- [19] LIU H, JIANG S, QI X, et al. Detect software vulnerabilities with weight biases via graph neural networks [J]. Expert Systems with Applications, 2024, 238: 121764.
- [20] WU Y M, ZOU D Q, DOU S H, et al. VulCNN: An image-inspired scalable vulnerability detection system [C]// ICSE 2022: Proceedings of the 44th International Conference on Software Engineering. New York: ACM, 2022: 2365-2376.
- [21] YAMAGUCHI F, GOLDE N, ARP D, et al. Modeling and discovering vulnerabilities with code property graphs [C]// S&P 2014: 2014 IEEE Symposium on Security and Privacy. Washington, DC: IEEE Computer Society, 2014: 590-604.
- [22] RIBEIRO L F R, SAVERESE P H P, FIGUEIREDO D R. struc2vec: Learning node representations from structural identity [C]// SIGKDD 2017: Proceedings of the 23rd ACM SIGKDD Conference on Knowledge Discovery and Data Mining. New York: ACM, 2017: 385-394.
- [23] GROVER A, LESKOVEC J. node2vec: Scalable feature learning for networks [C]// SIGKDD 2016: Proceedings of the 22nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining. New York: ACM, 2016: 855-864.
- [24] PAGLIARDINI M, GUPTA P, JAGGI M. Unsupervised learning of sentence embeddings using compositional n-gram features [C]// NAACL 2018: The 2018 Annual Conference of the North of the American Chapter of the Association for Computational Linguistics. Stroudsburg, PA: Association for Computational Linguistics, 2018: 528-540.
- [25] Zhang C, Xin Y. VulGAI: vulnerability detection based on graphs and images [J]. Computers & Security, 2023, 135: 103501.
- [26] ZHANG J, LUO Y. Degree centrality, betweenness centrality, and closeness centrality in social network [C]// MSAM 2017: Proceedings of the 2017 2nd International Conference on Modelling, Simulation and Applied Mathematics. Dordrecht: Atlantis press, 2017: 300-303.
- [27] Lin G, Xiao W, Zhang J, et al. Deep learning-based vulnerable function detection: A benchmark [C]// ICICS 2019: 21st International Conference on Information and Communications Security. Berlin: Springer International, 2020: 219-232.
- [28] CHENG X, WANG H Y, HUA J Y, et al. Static detection of control-flow-related vulnerabilities using graph embedding [C]// ICECCS 2019: 2019 24th International Conference on Engineering of Complex Computer Systems. Piscataway: IEEE, 2019: 41-50.
- [29] LI Z, ZOU D Q, XU S H, et al. VulDeeLocator: A deep learning-based fine-grained vulnerability detector [J]. IEEE Transactions on Dependable and Secure Computing, 2021, 19(4): 2821-2837.
- [30] ZHOU Y, LIU S, SIOW J, et al. Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks [J]. Advances in Neural Information Processing Systems, 2019, 32.

This work is partially supported by the National Natural Science Foundation of China (61902161), and the Postgraduate Research & Practice Innovation Program of Jiangsu Normal University (2024XKT 2616).

YU Qiao, born in 1989, Ph.D., associate professor. Her research interests include machine learning, software defect prediction, vulnerability detection.

HUANG Zirui, born in 1999, M. S. candidate. His research interests include vulnerability detection.

CHENG Shengyi, born in 1999, M. S. candidate. His research interests include vulnerability detection, software defect prediction.

ZHU Yi, born in 1976, Ph.D., professor. His research interests include software engineering, formal methods, software defect prediction.

ZHANG Shutao, born in 1979, M. S., senior engineer. His research interests include information management system and operations.

